

GINZA GAMING · SECURITY REVIEW

Ginza Gaming Review.

A security review of Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Ginza Gaming	Review	Guardian	October 8 – December 2, 2024	December 9, 2024

§ 00

Contents.

33 pages — overview, confidence ranking, scope, 23 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 16 findings	P. 09
	Remediation Review · 7 findings	P. 25
§ 05	Disclaimer	P. 32
§ 06	About Guardian	P. 33

§ 01

Overview.

Guardian reviewed Review across 2 review rounds. The final report includes 2 High, 3 Medium, 18 Low findings.

AUDIT FIRM	REPOSITORY
Guardian	GinzaGaming/ginza
CLIENT	METHODOLOGY
Ginza Gaming	Smart contract · Manual · Main Review · Remediation Review
PROJECT	MAIN REVIEW COMMIT
Review	c759b0ed37aa
PREPARED BY	
Guardian	
AUDIT WINDOW	
October 8 – December 2, 2024	
FINAL REPORT	
December 9, 2024	

1.1 Confidence Ranking

4 / 5

High Confidence

Guardian assigns a Confidence Ranking of 4 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	2	0	1	0	1
Medium	3	0	1	0	2
Low	18	0	9	0	9
Informational	0	0	0	0	0
All severities	23	0	11	0	12

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence ASSIGNED	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Review in `GinzaGaming/ginza`. The scope includes 354 source lines across 4 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
onchain/src/IERC20Rebasing.sol	4	9
onchain/src/IBlast.sol	12	17
onchain/src/GinzaStructs.sol	17	21
onchain/src/Ginza.sol	321	408

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
H-01	Automatic USDB Yields Are Trapped	Logical Error	HIGH	RESOLVED	p. 9
M-01	Tables With Too Many Players May Get Stuck	DoS	MEDIUM	RESOLVED	p. 10
M-02	Zero Address Player	Logical Error	MEDIUM	RESOLVED	p. 11
L-01	Arbitrary Players May Cancel	Validation	LOW	RESOLVED	p. 12
L-02	Anyone Can emergencyExit	Validation	LOW	RESOLVED	p. 13
L-03	Lacking UUPS Init Call	Best Practices	LOW	RESOLVED	p. 14
L-04	Tables Cannot Close	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 15
L-05	Lacking CEI In _cancelRequest	Best Practices	LOW	RESOLVED	p. 16
L-06	Lacking CEI In withdrawRake	Best Practices	LOW	RESOLVED	p. 17
L-07	safeTransferFrom Should Occur First	Best Practices	LOW	RESOLVED	p. 18
L-08	Useful Event Data	Best Practices	LOW	RESOLVED	p. 19
L-09	Lacking updateTableCredits Validation	Validation	LOW	ACKNOWLEDGED	p. 20
L-10	getAllPlayerRequestsForTable DoS	DoS	LOW	ACKNOWLEDGED	p. 21
L-11	User Leaves Twice	Events	LOW	ACKNOWLEDGED	p. 22
L-12	Frontrunning Protocol Share Change	Frontrunning	LOW	RESOLVED	p. 23

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-13	Player Can Top-up When Pending Removal	Logical Error	LOW	RESOLVED	p. 24

REMEDIATION REVIEW

H-01	Errant Credit Data Reported	Logical Error	HIGH	ACKNOWLEDGED	p. 25
M-01	Disable Initializers Removed	Best Practices	MEDIUM	ACKNOWLEDGED	p. 26
L-01	approveRequest Inefficient Seat Search	Optimization	LOW	ACKNOWLEDGED	p. 27
L-02	Seat Request Uncleared On Cancel	Best Practices	LOW	ACKNOWLEDGED	p. 28
L-03	Outdated Documentation	Documentation	LOW	ACKNOWLEDGED	p. 29
L-04	Yield Is Automatic By Default	Warning	LOW	ACKNOWLEDGED	p. 30
L-05	Unnecessary Stack Variable	Optimization	LOW	ACKNOWLEDGED	p. 31

HIGH **H-01**

RESOLVED

Automatic USDB Yields Are Trapped

CATEGORY Logical Error
LOCATION Ginza.sol: 123
REVIEW Main Review

DESCRIPTION

In the `initialize` function the USDB yields are set to automatic, however there is no mechanism to claim the yield which will be reflected in an increasing USDB balance for the contract.

RECOMMENDATION

Consider configuring `YieldMode.CLAIMABLE` and implementing a function to claim the USDB yields that have accrued for the contract. A similar approach could be implemented for the WETH token, though it is not expected that the contract holds WETH.

RESOLUTION

Status: Resolved.

MEDIUM **M-01**

RESOLVED

Tables With Too Many Players May Get Stuck

CATEGORY DoS
LOCATION Ginza.sol
REVIEW Main Review

— DESCRIPTION

There is currently no cap on how many players can be added to a table, and in the `updateTableCredits` function these players must be iterated over to remove players and create the `approved_players` and `player_credits` arrays.

These loops may cost an extreme amount of gas and even require more than the block gas limit to execute if there are many players in a single game. This could ultimately result in trapped funds for users, though they will be able to emergency withdraw.

— RECOMMENDATION

Consider capping the total amount of players which can be added to a table.

— RESOLUTION

Status: Resolved.

MEDIUM **M-02**

RESOLVED

Zero Address Player

CATEGORY Logical Error
LOCATION Ginza.sol: 164, 379
REVIEW Main Review

DESCRIPTION

A user can request a buy-in for the zero address. If accepted, this user cannot be kicked from the game. This is because `_removePlayerFromTable` will attempt to transfer `ERCUSDB` to the zero address, which reverts. Furthermore, if the user was added to the pending `playersToRemove`, the loop in `updateTableCredits` will revert causing DoS for the entire table. Ultimately, this player will never be able to get removed from the game.

RECOMMENDATION

Validate that the `player` is not `address(0)` in `requestBuyInOnBehalf`

RESOLUTION

Status: Resolved.

LOW **L-01**

RESOLVED

Arbitrary Players May Cancel

CATEGORY Validation
LOCATION Ginza.sol: 172
REVIEW Main Review

DESCRIPTION

In the `_cancelRequest` function there is no validation to ensure that the player has an active request to cancel for the table.

Thus a user can cause the `PlayerCancelRequest` event to be emitted even if they did not have a pending request. This may cause unexpected behavior for frontend systems or other consumers of the events from the Ginza contract.

RECOMMENDATION

Consider reverting with a descriptive custom error if the `table.playerRequest.remove(player)` operation returns false. Or performing a dedicated validation first thing in the function.

RESOLUTION

Status: Resolved.

<https://github.com/GinzaGaming/ginza/commit/5e2fbdc69aa244e54d4c0d3317acd9009a348e9a>

LOW **L-02**

RESOLVED

Anyone Can emergencyExit

CATEGORY	Validation
LOCATION	Ginza.sol: 365
REVIEW	Main Review

DESCRIPTION

The `emergencyExit` function does not have an `onlyApprovedPlayer` modifier, therefore any arbitrary player can call the `emergencyExit` function on a stale table.

Currently there is no effect beyond emitting the `PlayerLeft` event which may affect off-chain systems.

RECOMMENDATION

Consider adding an `onlyApprovedPlayer` modifier to the `emergencyExit` function.

RESOLUTION

Status: Resolved.

<https://github.com/GinzaGaming/ginza/commit/1290b356df714a2985403fee34af8887d14a55c0>

LOW **L-03**

RESOLVED

Lacking UUPS Init Call

CATEGORY	Best Practices
LOCATION	Ginza.sol
REVIEW	Main Review

DESCRIPTION

In the initialize function in the Ginza contract there is no invocation of the `__UUPSUpgradeable_init` function. There is no logic in the `__UUPSUpgradeable_init` function in the `UUPSUpgradeable` contract, however it is a best practice to call this function if this were to change in a future iteration.

RECOMMENDATION

Consider calling the `__UUPSUpgradeable_init` function in the `initialize` function.

RESOLUTION

Status: Resolved.

<https://github.com/GinzaGaming/ginza/commit/402956f30b26beb847246250fc2f44e2c89ec702>

LOW **L-04**

ACKNOWLEDGED

Tables Cannot Close

CATEGORY	Unexpected Behavior
LOCATION	Ginza.sol
REVIEW	Main Review

— DESCRIPTION

In the Ginza contract tables can only be opened and not closed.

— RECOMMENDATION

Consider if this is the expected behavior. If not, consider implementing a function which that allows the table owner to close the table only after all players have left.

— RESOLUTION

Status: Acknowledged.

LOW **L-05**

RESOLVED

Lacking CEI In `_cancelRequest`

CATEGORY Best Practices

LOCATION `Ginza.sol: 182`

REVIEW Main Review

DESCRIPTION

In the `_cancelRequest` function the `table.playerRequest.remove(player)` action is performed after transferring tokens to the user. However this accounting update ought to occur before giving the user their tokens to follow best practices.

RECOMMENDATION

Consider performing the `table.playerRequest.remove(player)` logic before transferring tokens to the user.

RESOLUTION

Status: Resolved.

<https://github.com/GinzaGaming/ginza/commit/fe930ca771dc3dda92fa40dbb95d59c624ff82b2>

LOW **L-06**

RESOLVED

Lacking CEI In withdrawRake

CATEGORY Best Practices
LOCATION Ginza.sol
REVIEW Main Review

DESCRIPTION

In the `withdrawRake` function the `collectedRake` entry for the user is reset to zero after the `safeTransfer` call sends their collected amount to the user. This is not immediately a risk as USDB is not a callback token, however it is a best practice to follow CEI for transfers to a user.

RECOMMENDATION

Consider updating all accounting before making transfers to the user.

RESOLUTION

Status: Resolved.

<https://github.com/GinzaGaming/ginza/commit/e3a9c3b93201c0eb288611a99e4094878229453c>

LOW **L-07**

RESOLVED

safeTransferFrom Should Occur First

CATEGORY	Best Practices
LOCATION	Ginza.sol
REVIEW	Main Review

— DESCRIPTION

In the `playerTopUp` and `_requestBuyIn` functions the `safeTransferFrom` action occurs after all accounting updates have occurred.

However it is a best practice to transfer funds in before performing state updates. This is because it is a valid state for the protocol to have received funds and not made accounting updates, but not for the protocol to have made accounting updates and not received funds.

— RECOMMENDATION

Perform `safeTransferFrom` after all initial validations in a function, but before any of the accounting updates are performed in the `playerTopUp` and `_requestBuyIn` functions.

— RESOLUTION

Status: Resolved.

<https://github.com/GinzaGaming/ginza/commit/a6c30c1595ecfa6628bb30412e9902e3f41851fb>

LOW **L-08**

RESOLVED

Useful Event Data

CATEGORY Best Practices
LOCATION Ginza.sol: 183
REVIEW Main Review

— DESCRIPTION

In the `PlayerCancelRequest` a useful piece of information may be who is initiating the cancellation, as either the user or the table owner may do so.

— RECOMMENDATION

Consider if the canceler is a useful piece of information to add to the `PlayerCancelRequest` event.

— RESOLUTION

Status: Resolved.

<https://github.com/GinzaGaming/ginza/commit/721599dc6666035255f8f4b7375b254a82a61a57>

LOW **L-09**

ACKNOWLEDGED

Lacking updateTableCredits Validation

CATEGORY Validation
LOCATION Ginza.sol: 229
REVIEW Main Review

DESCRIPTION

In the `updateTableCredits` function there is no validation that a user is not in both of the winners and losers lists.

RECOMMENDATION

Consider implementing validations such that no user may appear in both the winner and loser lists.

RESOLUTION

Status: Acknowledged.

LOW **L-10**

ACKNOWLEDGED

getAllPlayerRequestsForTable DoS

CATEGORY DoS
LOCATION Ginza.sol: 453
REVIEW Main Review

— DESCRIPTION

The `getAllPlayerRequestsForTable` function iterates over all of the pending player requests to return the total player requests and amounts. There is no limit on how many player requests there are for a table, and there shouldn't be, otherwise users could block preferred players from joining a game.

However this means that the amount of player requests for a table may be extensive, even enough to overflow the block gas limit. For this reason the `getAllPlayerRequestsForTable` function should be used for off-chain reading purposes only and no Smart Contract should attempt to integrate it into its own logic.

— RECOMMENDATION

Be aware of this risk and warn potential integrators about it.

— RESOLUTION

Status: Acknowledged.

LOW **L-11**

ACKNOWLEDGED

User Leaves Twice

CATEGORY	Events
LOCATION	Ginza.sol
REVIEW	Main Review

— DESCRIPTION

A user can be kicked with function `kickPlayer` which will create a removal request and emit a `PlayerLeave` event. If all other players are removed, the user can be kicked again and now will be directly removed from the table with `_removePlayerFromTable`. This will also emit a `PlayerLeave` event.

This may cause issues with the UI relying on these events, since it appears as if the player left twice without joining in between.

— RECOMMENDATION

Be aware of this behavior and consider removing a user from `playersToRemove` if he was already directly removed with `_removePlayerFromTable`.

— RESOLUTION

Status: Acknowledged.

LOW **L-12**

RESOLVED

Frontrunning Protocol Share Change

CATEGORY Frontrunning
LOCATION Ginza.sol
REVIEW Main Review

— DESCRIPTION

It is possible for a user to frontrun a call to `setProtocolShare` which would increase the share, and `withdrawRake` right before to pay the smaller fee.

— RECOMMENDATION

Be aware of this behavior and document it appropriately.

— RESOLUTION

Status: Resolved.

LOW **L-13**

RESOLVED

Player Can Top-up When Pending Removal

CATEGORY Logical Error
LOCATION Ginza.sol: 209
REVIEW Main Review

DESCRIPTION

A user can call function `playerTopUp` to increase their credit even if they are in the `playersToRemove` list because they are still considered active until removal. This may be unexpected from a user's perspective.

RECOMMENDATION

Consider if this is intended behavior and document as necessary.

RESOLUTION

Status: Resolved.

HIGH **H-01**

ACKNOWLEDGED

Errant Credit Data Reported

CATEGORY Logical Error
LOCATION Ginza.sol: 661
REVIEW Remediation Review

DESCRIPTION

The `chainSync` function returns table data in the order of approvedPlayers, credits, seats, pendingPlayers, and requestedAmounts. However the `batchChainSync` function stores the returned values as approvedPlayers, seats, credits, pendingPlayers, and requestedAmounts where the seats and credits values are switched.

This will mistaken the players seat number for their credit amount and cause significant loss for the players using the system relying on this reporting.

RECOMMENDATION

Correct the ordering of the values in the `batchChainSync` to agree with the ordering of the `chainSync` function.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

Disable Initializers Removed

CATEGORY Best Practices
LOCATION GinzaV4.sol
REVIEW Remediation Review

DESCRIPTION

In the `GinzaV4` contract the constructor with the `_disableInitializers` invocation has been removed. This is likely for testing purposes but the function call should be re-instated before deployment to avoid a malicious upgrade and self destruction of the implementation UUPS contract.

RECOMMENDATION

Re-introduce the constructor with the `_disableInitializers` call in all Ginza contracts before production use.

RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

approveRequest Inefficient Seat Search

CATEGORY Optimization
LOCATION Ginza.sol: 268
REVIEW Remediation Review

DESCRIPTION

In the `approveRequest` function the new player is added to the `approvedPlayers` list before the `findNextAvailableSeat` operation. The `findNextAvailableSeat` operation uses the `approvedPlayers` list to find each taken seat to deduce if a seat is available or not.

Since the new player is added to the `approvedPlayers` list before the `findNextAvailableSeat` operation the `findNextAvailableSeat` function must add an additional unnecessary iteration for this new player for every seat that is checked.

RECOMMENDATION

Consider adding the new player to the `approvedPlayers` list after the `findNextAvailableSeat` operation to save on gas expenditure when approving a player.

RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Seat Request Uncleared On Cancel

CATEGORY Best Practices
LOCATION GinzaV4.sol: 204
REVIEW Remediation Review

DESCRIPTION

When creating a request to join a table the desired seat is recorded in the `seatAssignments` mapping, however in the `_cancelRequest` function the user is not removed from the `seatAssignments` mapping.

RECOMMENDATION

Remove all remnants of the player's request from the `seatAssignments` mapping in the `_cancelRequest` function.

RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

Outdated Documentation

CATEGORY	Documentation
LOCATION	GinzaV4.sol
REVIEW	Remediation Review

DESCRIPTION

In the `GinzaV4` contract the `emergencyWithdraw` threshold has been reduced to 15 minutes, however the documentation in the contract suggests that the threshold is still 24 hours.

RECOMMENDATION

Update the documentation to reflect the new threshold.

RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

Yield Is Automatic By Default

CATEGORY Warning
LOCATION GinzaV4.sol: 121
REVIEW Remediation Review

DESCRIPTION

In the `initialize` function the USDB yields are configured to automatic by default, however the functionality which exists to claim these yields is using a claimable yield mode.

RECOMMENDATION

Instead of including the `makeYieldClaimable` function, consider making the yield claimable by default in the `initialize` function.

RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

Unnecessary Stack Variable

CATEGORY Optimization
LOCATION GinzaV4.sol: 693
REVIEW Remediation Review

DESCRIPTION

In the `batchChainSync` function the `blockNumber_` stack variable is declared and never used.

RECOMMENDATION

Remove the declaration of the `blockNumber_` stack variable and instead leave a blank comma for the final return value from the `chainSync` function.

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.